Curs 3 - Calcularea Complexității Algoritmilor Recursivi (1)

2024

Cuprins

1 Introducere **1**

2 MergeSort **1**
2.1 Implementare 1

3 Metoda Iterației **2**
3.1 MergeSort 2
3.2 Turnurile din Hanoi 2
3.3 Exerciții propuse 3

4 Metoda Arborelui de Recurență [1] **3**
4.1 MergeSort 3
4.2 Arbore dezechilibrat 4
4.3 Exerciții propuse 5

1 Introducere

Primii pași în analiza complexității temporale a unui algoritm recursiv sunt identificarea:

- Dimensiunii cazurilor de bază, care pot fi rezolvate direct, fără apeluri recursive.
- Efortului de calcul necesar descompunerii problemei inițiale în subprobleme mai mici.
- Dimensiunilor subproblemelor rezolvate recursiv.
- Efortului de calcul necesar unificării rezultatelor subproblemelor.

În acest curs ne propunem să studiem aplicabilitatea a două metode pentru analiza complexității unor algoritmi recursivi, metoda iterației, respectiv metoda arborelui de recurență.

2 MergeSort

2.1 Implementare

```
1: function MergeSort(A[1..n], start, end)
2:     if start ≥ end then return
3:     end if
4:     middle = (start + end)/2          // Divide - θ(1)
5:     MergeSort(A, start, middle)       // Impera: Sortează recursiv prima jumătate a secvenței - T(⌊N/2⌋)
6:     MergeSort(A, middle + 1, end)     // Impera: Sortează recursiv a doua jumătate a secvenței - T(⌈N/2⌉)
7:     Merge(A, start, middle, end)      // Interclasează rezultatele celor două apeluri recursive - θ(N)
8: end function
```

Pe moment, pentru a simplifica analiza, vom considera că cele două subprobleme recursive au aceeasi dimensiune pentru fiecare apel, respectiv $\frac{n}{2}$[1], unde n reprezintă dimensiunea secvenței primită la intrare. De asemeni, observăm că pasul de interclasare poate fi încadrat în $\theta(n)$. Prin urmare, putem descrie complexitatea temporală folosind următoarea relație de recurență:

$$T(n) = 2 \cdot T(\frac{n}{2}) + c \cdot n; T(1) = 1. \tag{1}$$

3 Metoda Iterației

3.1 MergeSort

1. Recurența originală: $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + c \cdot n;\ T(1) = c$
2. Explicităm termenul recursiv: $T\left(\frac{n}{2}\right) = 2 \cdot T\left(\frac{n}{4}\right) + c \cdot \frac{n}{2}$
3. Înmulțim cu factorul asociat: $2 \cdot T\left(\frac{n}{2}\right) = 2^2 \cdot T\left(\frac{n}{4}\right) + c \cdot n$
4. Repetăm până intuim forma generală pentru termenul liber.

$$\begin{aligned}
T(n) &= 2 \cdot T\left(\frac{n}{2}\right) && +c \cdot n \\
2 \cdot T\left(\frac{n}{2}\right) &= 4 \cdot T\left(\frac{n}{4}\right) && +c \cdot n \\
4 \cdot T\left(\frac{n}{4}\right) &= 8 \cdot T\left(\frac{n}{8}\right) && +c \cdot n \\
&\dots \\
2^k \cdot T\left(\frac{n}{2^k}\right) &= 2^{k+1} \cdot T\left(\frac{n}{2^{k+1}}\right) && +c \cdot n \\
&\dots \\
2^{h-1} \cdot T\left(\frac{n}{2^{h-1}}\right) &= 2^h \cdot \underbrace{T\left(\frac{n}{2^h}\right)}_{T(1)} && +c \cdot n
\end{aligned}$$

5. Pentru a afla adâncimea apelurilor recursive, h, trebuie egalat $T(1)$ cu forma generală găsită $T\left(\frac{n}{2^h}\right)$:

$$\frac{n}{2^h} = 1 \Rightarrow h = \log_2(n)$$

6. Explicităm pentru cazul de bază $T(1)$: $2^h \cdot T(1) = n \cdot c$
7. Adunăm toți termenii liberi:

$$T(n) = h \cdot c \cdot n + n \cdot c = (\log_2(n) + 1) \cdot c \cdot n$$

8. $T(n) \in \Theta(n \cdot \log(n))$

3.2 Turnurile din Hanoi

1. Recurența originală:
   $T(n) = 2 \cdot T(n-1) + c \cdot n;\ T(1) = c$
2. Explicităm termenul recursiv:
   $T(n-1) = 2 \cdot T\left(\frac{n}{2}\right) + c$
3. Înmulțim cu factorul asociat:
   $2 \cdot T(n-2) = 2^2 \cdot T(n-2) + 2 \cdot c$

[1] Cu alte cuvinte, presupunem că n este o putere a lui 2.

4. Ultimul apel, înainte de cazul de bază:
   $2^{(h-1)}\ T(n-(h-1)) = 2^h \cdot T(n-h) + 2^{h-1} \cdot c$
5. Numărul de apeluri recursive până atingem cazul de bază (T(1)):
   $n - h = 1 \implies h = n - 1$
6. Adunăm toți termenii:
   $T(n) = c * (2^0 + 2^1 + 2^2 + \cdots + 2^{h-1} + 2^h)$
   $T(n) = c * (2^{h+1} - 1)$
   $T(n) \in \Theta(2^n)$

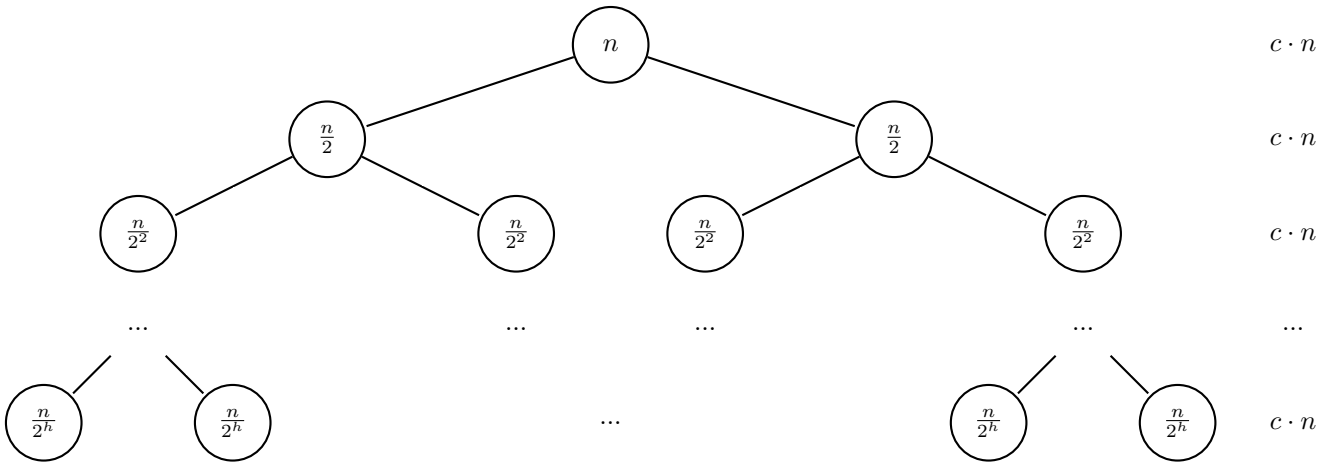
3.3 Exerciții propuse

- $T(n) = T(n-k) + c \cdot n$, $T(1) = 1$.
- $T(n) = 3 \cdot T(\frac{n}{2}) + n^c$, $T(1) = 1$.

4 Metoda Arborelui de Recurență [1]

O alternativă la metoda iterației este să reprezentăm grafic arborele de recurență. Pentru fiecare apel recursiv vom desena un nod, pentru care vom scrie dimensiunea problemei rezolvate în cadrul respectivului apel. Astfel, putem vizualiza mai ușor cum se distribuie efortul de calcul pentru fiecare nivel din recursivitate și să intuim forma generală a sumei. În cursurile următoare vom vedea cum putem valida formal această intuiție, folosind inducție.

4.1 MergeSort



$$\sum_{i=0}^{h} c \cdot n = h \cdot c \cdot n \in \theta(n * log(n))$$

Figura 1: Reprezentare grafică a arborelui de recurență corespunzător algoritmului MergeSort. În fiecare nod din partea stângă am scris dimensiunea subproblemei asociate. Pe partea din dreapta am scris suma efortului de calcul corespunzător fiecărui nivel. De exemplu, pentru nivelul 1, avem două subprobleme de dimensiune $\frac{n}{2}$. Suma acestora este $c \cdot \frac{n}{2} + c \cdot \frac{n}{2} = c \cdot n$. Înălțimea arborelui este dată de numărul maxim de apeluri necesare până atingem cazul de bază ($T(1)$), respectiv $log_2(n)$.

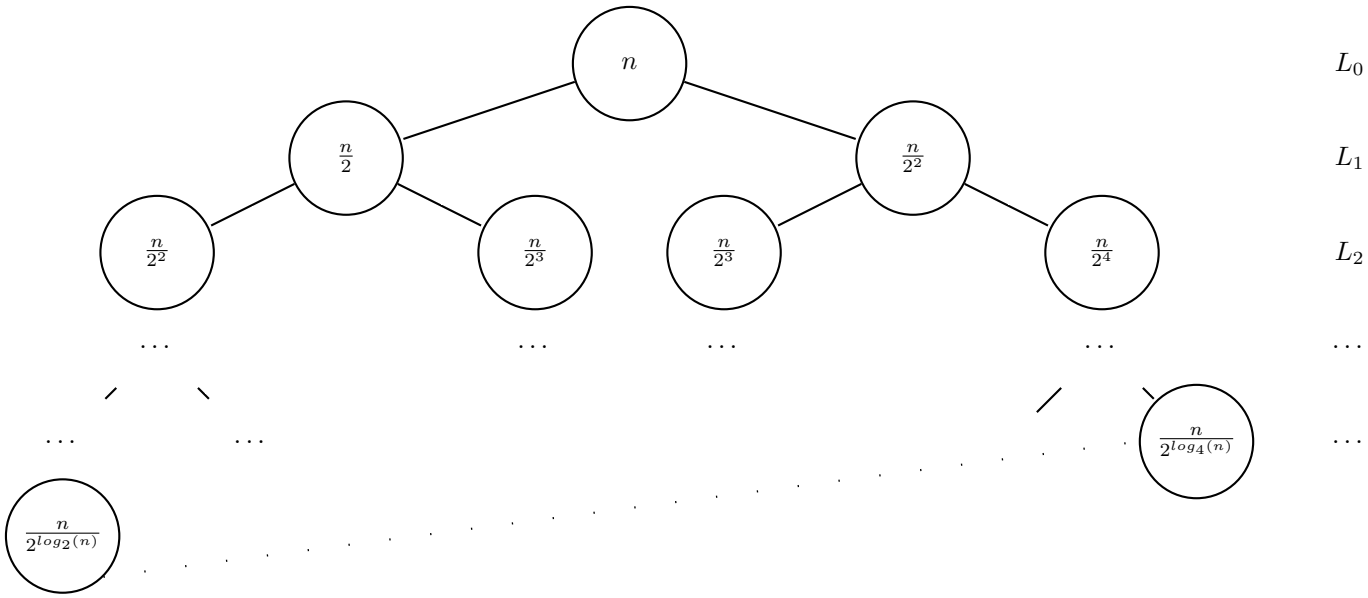


Figura 2: Reprezentare grafică a arborelui de recurență corespunzător relației de recurență $T(n) = T(\frac{n}{2}) + T(\frac{n}{4}) + c * n^2$.

4.2 Arbore dezechilibrat

În particular, această metodă este mai ușor de aplicat, față de metoda iterației, pentru arbori dezechilibrați, în care subproblemele de pe fiecare nivel au dimensiuni diferite. Fie următoarea recurență (v. Fig. 2):

$$T(n) = T(\frac{n}{2}) + T(\frac{n}{4}) + c \cdot n^2;\ T(1) = 1$$

Pentru primele nivele din recursivitate anticipăm următorul efort de calcul:

$$\begin{aligned}
&L_0 = c \cdot n^2 \\
&L_1 = c \cdot ((\frac{n}{2})^2 + (\frac{n}{2^2})^2) = c \cdot n^2 (\frac{1}{2^2} + \frac{1}{2^4}) = c \cdot n^2 \cdot \frac{5}{16} \\
&L_2 = c \cdot ((\frac{n}{2^2})^2 + 2 \cdot (\frac{n}{2^3})^2 + (\frac{n}{2^4})^2) = c \cdot n^2 \cdot (\frac{1}{2^4} + \frac{1}{2^6} + \frac{1}{2^8}) = c \cdot n^2 \cdot (\frac{5}{16})^2 \\
&\dots \\
&L_i = c \cdot n^2 \cdot (\frac{5}{16}^{\ i})\ \text{(pentru } i \leq log_4(n))
\end{aligned}$$

Înălțimea maximă a arborelui de recurență este tot $log_2(n)$. Observăm însă că nu avem, de data aceasta, un arbore binar complet. Dacă la recurența precedentă, pentru fiecare nivel L_i întâlneam exact 2^i noduri, de data aceasta, începând cu nivelul $L_{log_4(n)+1}$ numărul de noduri începe să scadă sub această valoare. Astfel, pe ultimul nivel vom avea chiar un singur nod.

În acest punct, o posibilă abordare este să identificăm expresii care să mărginească inferior, respectiv superior, valoarea lui $T(n)$.

$$c \cdot n^2 \cdot \sum_{i=0}^{log_4(n)} (\frac{5}{16})^i \leq T(n) \leq c \cdot n^2 \cdot \sum_{i=0}^{log_2(n)} (\frac{5}{16})^i \tag{2}$$

Observăm că avem de calculat suma unei progresii geometrice a cărei rație este subunitară:

$$S = \sum_{i=0}^{h} x^i = \frac{1 - x^{h+1}}{1 - x}$$

$$1 \leq S < \sum_{i=0}^{\infty} x^i = \frac{1}{1 - \frac{5}{16}} = \frac{16}{11}$$

Înlocuind acest rezultat în relația 2 obținem:

$$c \cdot n^2 \leq T(n) \leq c * \frac{16}{11} \cdot n^2$$

Prin urmare, $T(n) \in \theta(n^2)$.

4.3 Exerciții propuse

- $T(n) = T(\frac{n}{5}) + T(7 * \frac{n}{10}) + n$.
- $T(n) = T(\frac{n}{2}) + T(\frac{n}{4}) + T(\frac{n}{8}) + n$.
- $T(n) = 2 \cdot T(\frac{n}{4}) + 1$.
- $T(n) = 5 \cdot T(\frac{n}{4}) + c \cdot n$.

Follow-up 1:

Cum se modifică complexitatea algoritmului MergeSort dacă alegem un raport diferit de spargere a problemei inițiale în două subprobleme (e.g. $T(\frac{n}{3})$ și $T(\frac{2n}{3})$)?

Follow-up 2:

Cum putem interclasa eficient m secvențe sortate, fiecare secvență având aceeași dimensiune (n)? Ce complexitate obținem dacă alegem să le interclasăm pe rând: prima secvență cu a doua, rezultatul obținut cu a treia, etc.?

References

[1] Charles E Leiserson et al. *Introduction to algorithms*. MIT press, 2022.